

Fraud EDA, Explained: Features, Results, and Plots

Sep 28, 2026 · @Salamot Fakoya

Before we start

Short version: fraud here depends on **which account is paying, which product it is, and which data is missing**. Everything else is smaller, and a lot of it turns out to be product type in disguise.

This is the friendly walkthrough of `BTT_Mastercard_EDA_Combined.ipynb`. It explains the features we built, what we found, and how to read each plot.

How to use this guide. Look for the **Find it:** lines. Each one quotes the comment or first line of a notebook cell. Press Ctrl+F (Cmd+F on Mac) in the notebook, paste it, and you land on the code.

Want exact numbers and code? The companion Markdown notes (`EDA_Summary_Notes.md`) go deeper, and they open on GitHub, VS Code or Colab without any setup. They cover:

- **Setup:** loading, the memory downcast, and the time split.
- **Every engineered feature:** its code snippet, a plain explanation, and whether it's a model feature or analysis only. That includes how the account key now handles missing addresses.
- **Helper functions** like `category_rr` and `entity_purity`, since you'll keep running into them.
- **Results by branch**, mostly as tables, including how the 89% is calculated, why it rose from 77% after the missing-address fix, and what it does and doesn't cover.
- **Modeling decisions, known weaknesses, and a glossary** of the statistics (relative risk, confidence intervals, Benjamini-Hochberg, silhouette score).

Each section of the notes quotes the comment that starts its notebook cell, like `# D0 (B4-H2): ...`, so searching the notebook for it takes you straight to the code.

The data in one breath. About 590,000 online card transactions over roughly six months. Each is labeled fraud or legit, and only 3.5% are fraud. Most of the 434 columns are masked, so we often know a column matters without knowing what it measures.

How the notebook is organized. It has four branches, each asking one question:

1. **Target and time:** how rare is fraud, and does it change over time?
2. **Data quality:** what's missing, and does missingness mean anything?
3. **Feature signal:** do amount, time, card, email and device predict fraud?
4. **Entity structure:** does fraud cluster around particular accounts?

Each hypothesis runs through Plan, Do, Check, Act. The code cells start with comments like `# D0 (B4-H2)`, meaning Branch 4, Hypothesis 2. Search the notebook for those to jump to the code.

One thing to know first. Before any exploring, we set aside the last 20% of the training data (by time) as validation and never looked at it. That keeps our validation scores honest later. So every number below comes from the first 80%: 472,432 transactions.

Loading the data

The data comes as two tables joined on `TransactionID`: transactions (amount, product, card, email, the `C`, `D`, `M`, `V` columns) and identity (device and browser info, only for about a quarter of transactions).

Find it: `# Test identity uses id-01 style names; align with train's id_01`

```
test_identity.columns = test_identity.columns.str.replace("-", "_")
train = train_transaction.merge(train_identity, on="TransactionID",
                                how="left")
```

`how="left"` keeps every transaction, even without identity data. The first line fixes a naming mismatch: the test file says `id-01`, training says `id_01`.

Find it: `# (from Eric's EDA) Downcast float64 columns to float32`. This stores decimal columns in half the memory; `TransactionAmt` is skipped so its exact cents survive.

The time split

Find it: `# Time-based split: last 20% of transactions by time from train set become validation`

```
train = train.sort_values("TransactionDT").reset_index(drop=True)
cutoff = int(len(train) * 0.8)
train_df = train.iloc[:cutoff].copy()
val_df = train.iloc[cutoff:].copy()
```

Set	Days	Transactions	Fraud rate
train_df	1 to 141	472,432	3.51%
val_df	141 to 182	118,108	3.44%
Kaggle test (no labels)	213 to 395	506,691	unknown

Find it: # (from Eric's EDA) Where does each period fall in time?. That cell found that test starts **30 days** after our labeled data ends, while validation starts the very next day. So validation scores may look a bit better than test scores will.

The features we built

We created about a dozen new columns and objects. Some are meant for the model; others only helped us understand the data. Here's each one in plain words.

Golden rule: anything learned from data (a grouping, a median, a cluster, a list of columns to keep) is computed on training data only, then reused unchanged on validation and test. Otherwise the future leaks into training.

Time: day , hour , dow

Find it: decision_log = [] (the first cell of Branch 1)

```
train_df["day"] = (train_df["TransactionDT"] // 86400).astype(int)
train_df["hour"] = (train_df["TransactionDT"] // 3600 % 24).astype(int)
train_df["dow"] = train_df["day"] % 7
```

TransactionDT is seconds from an unknown start, not a real date. We divide it into a **day number**, an **hour of the day** (0 to 23) and a **day of week** (0 to 6, though we don't know which is Monday).

hour turned out to be one of the strongest signals. day is not a model feature, because the test set covers days the model never saw.

has_identity

Find it: # D0 (H3): identity coverage and fraud rate with vs without identity

```
id_cols = [c for c in train_df.columns if c.startswith("id_")] +
["DeviceType", "DeviceInfo"]
train_df["has_identity"] = train_df[id_cols].notna().any(axis=1).astype(int)
```

A simple yes/no: does this transaction have any identity or device data? Only about a quarter do. It's useful, but it overlaps heavily with product type, since product **W** never has identity data.

Missingness blocks

Find it: # D0 (H2): group columns by identical null count to find missingness blocks. The two cells after it confirm the rows really match, not just the counts.

We noticed many columns go empty on **exactly the same rows**. We call those groups **blocks**. A block probably means one data source that either reported everything or nothing. We found 24 of them.

This matters because one "is missing" flag per block is enough, and it sets up the next feature.

v_keep : trimming the V columns

Find it: # D0 (H2 follow-up): near-duplicate V columns within each missingness block, train_df only

The 339 **V** columns are anonymous, and many are near-copies of each other. Inside each block, we compared every pair and dropped any column that was more than 95% correlated with one we already kept. We favored the column with the most distinct values, since it carries the most detail.

Step by step, inside each block:

1. Take a 100,000-row sample of training data.
2. Measure how closely every pair of V columns moves together (Spearman correlation).
3. Start with the column that has the most distinct values.
4. Keep each next column only if it's not more than 95% correlated with one already kept.

Result: 210 of 339 kept. That's the V set for simpler models like logistic regression.

Missingness profiles and clusters

Find it: # One indicator per missingness pattern (1 = missing), from the H4 table for the profiles, and `km = MiniBatchKMeans(n_clusters=best_k, ...)` for the clusters (`na_cluster`).

For each transaction we wrote down which of the 67 missing-data patterns it has: a string of 67 ones and zeros. That's its **profile**. There are 11,341 different profiles, but just 118 cover 80% of transactions.

We then ran **k-means clustering** on those profiles to find natural groups. It found three, without being told about product or identity. We also flag each transaction as a **common** or **rare** profile.

Email providers and email match

Find it: # D0 (B3-H4a): fraud rate by purchaser email domain and provider and # D0 (B3-H4b): does the recipient domain match the purchaser domain?

```
train_df["P_provider"] = train_df["P_emaildomain"].str.split(".").str[0]
```

`P_provider` keeps just the part of the email domain before the first dot, so `yahoo.com` and `yahoo.fr` both become `yahoo`. That turns 59 domains into 44 providers.

`email_match` sorts each transaction into: no recipient, no purchaser, same domain, or different domain.

device_family

Find it: # D0 (B3-H5): device information among transactions with identity data, then the second `def device_family(s):` cell, which adds more rules.

`DeviceInfo` has 1,639 messy strings like `SM-G930V Build/NRD90M`. We match keywords in order (`sm-` means Samsung, `moto` means Motorola) to sort them into 17 families. Unmatched Android strings become `android_other`.

The account key: D1n and uid

Find it: # D0 (B4-H1): build entity keys and test whether day - D1 is stable within `card1 + addr1`

```

train_df["D1n"] = train_df["day"] - train_df["D1"]
no_addr = train_df["addr1"].isna()
addr_key = train_df["addr1"].astype(str).where(~no_addr, "noaddr" +
train_df["TransactionID"].astype(str))
train_df["card_addr"] = train_df["card1"].astype(str) + "_" + addr_key
train_df["uid"] = train_df["card_addr"] + "_" + train_df["D1n"].astype(str)

```

The data has no account ID, so we built one. **D1** seems to count days since a card was first used. So **D1n = day minus D1** should be the card's start day, and stay the same across all its purchases.

For example, a card that started on day 150 and is used on days 160, 175 and 190 has **D1** values of 10, 25 and 40. **D1n** is 150 every time.

Then **uid = card identifier + billing address + D1n**. That's our best guess at an account. We don't feed **uid** directly to the model (it would just memorize accounts), but we can summarize each account's behavior.

One detail: if a transaction's address is missing, it gets its own unique marker instead of an address. That way, unrelated purchases with no address aren't merged into one fake account.

Analysis-only helpers

A few things exist only to make charts or tables: **log_amt** (log of the amount, to tame skew), amount bands (Eric's price buckets), and PCA coordinates (to plot the profiles in 2D).

Helper functions you'll keep seeing

Three small functions do a lot of the work across the notebook.

- **log(branch, finding, decision)** adds a row to **decision_log**, our running record of findings and decisions. **Find it:** `pd.DataFrame(decision_log)` in the last cell prints the whole log.
- **category_rr(data, col)** compares each value of a column (say, each card network) against everyone else: its fraud rate, the ratio between the two, a 95% confidence interval, and whether it has at least 1,000 transactions. **Find it:** `# D0 (B3-H3): fraud rate by card network and card type`

- **entity_purity(data, key)** groups transactions into accounts and measures how pure they are (all legit, all fraud, or mixed). With a `seed`, it shuffles the fraud labels first so we can compare against chance. **Find it:** # D0 (B4-H2): does fraud cluster within entities?

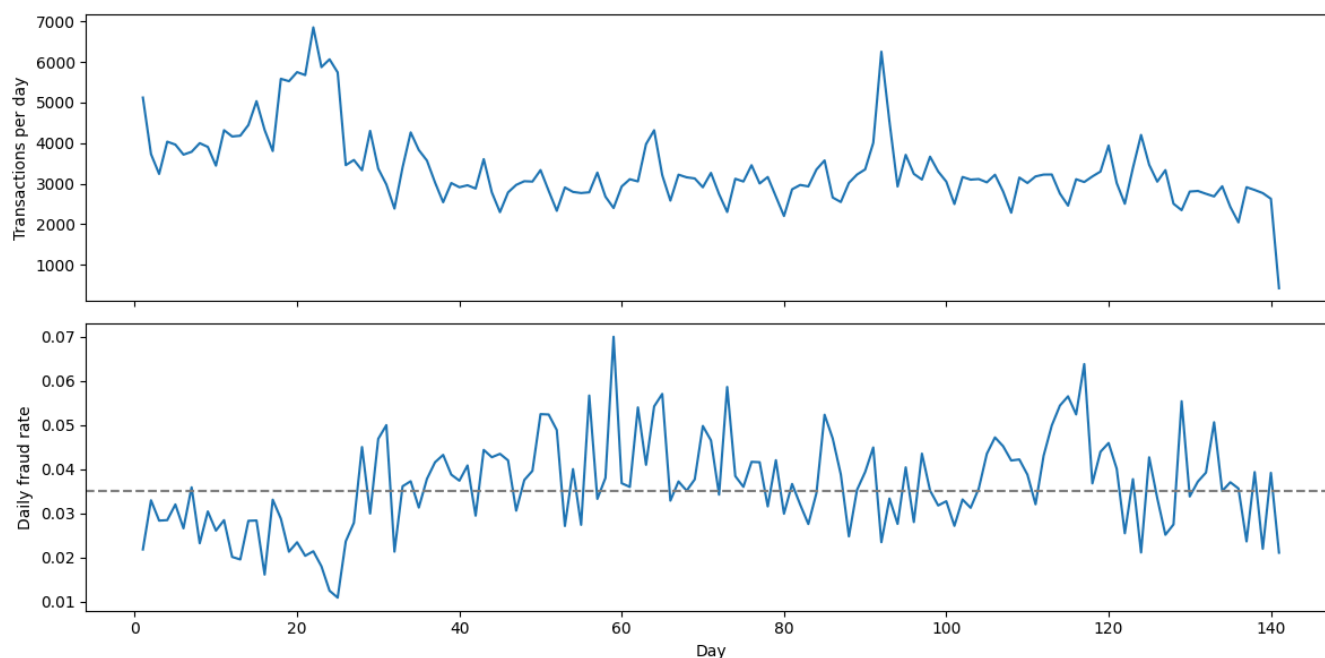
Branch 1: How rare is fraud, and does it change over time?

Fraud is rare (3.5%) and it shifts over time, so we judge models with the right metrics and keep a time-based split.

Find it: `decision_log = []` (the first cell of Branch 1)

Rare. 16,599 of 472,432 training transactions are fraud. A model that always says "legit" would be 96.5% accurate and catch nothing. So we use **ROC-AUC** and **PR-AUC**, never accuracy.

Shifting. The first 25 days have 2.39% fraud; after that it's 3.88%. The early period has about 40% less fraud (relative risk 0.62, 95% CI 0.59 to 0.64).



How to read it. Top panel: transactions per day. Bottom panel: the share of each day's transactions that were fraud; the dashed line is the 3.51% average.

- Days 17 to 25 are the busiest (up to about 6,800 a day) and have the lowest fraud. Maybe holiday shopping diluting fraud, though we don't know the real dates.
- After about day 27, fraud sits higher and jumps around more (peaks near day 59 and day 117).

- The cliff at day 141 isn't real: our train/validation split cut that day in half.

So what? Raw `day` can't be a feature (the test period is months later), and a random split would hide this drift. The Kaggle test set also starts **30 days** after our data ends, so we may add a gap between training and validation when modeling.

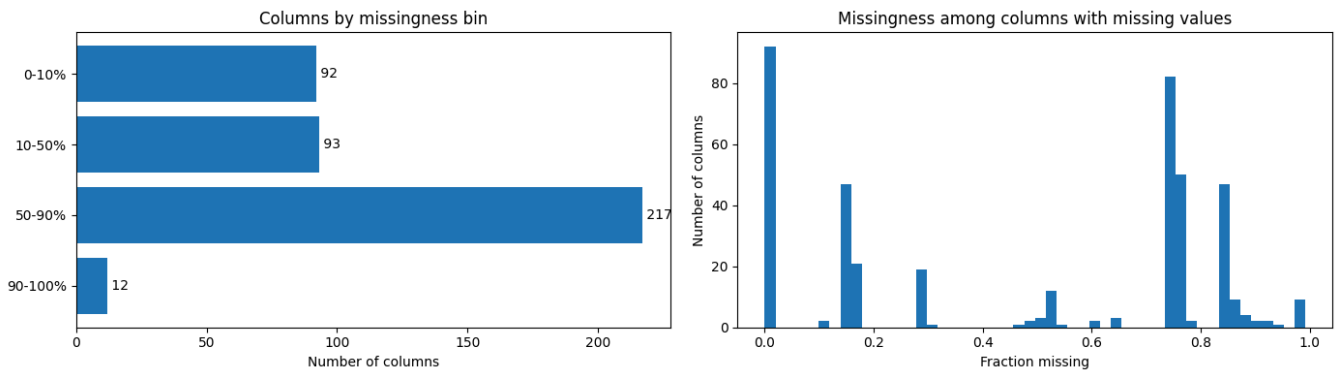
Branch 2: What's missing, and does it mean anything?

Missing data is everywhere, it's organized, and it predicts fraud. It also revealed that **product type** shapes almost everything.

Lots of gaps, in neat blocks

Find it: `# D0 (H1): missingness per column, in non-overlapping bins`

414 of 434 columns have missing values, and 229 are more than half empty. But the gaps aren't random: the columns fall into **24 blocks** that go missing on exactly the same rows.



How to read it. Left: how many columns fall in each missing-rate bucket. Right: each column's missing fraction. Notice the tall spikes at a few exact levels (around 15%, 28%, 52%, 75% and 85%). Many columns sharing one exact rate was our first clue that blocks exist.

Missingness predicts fraud

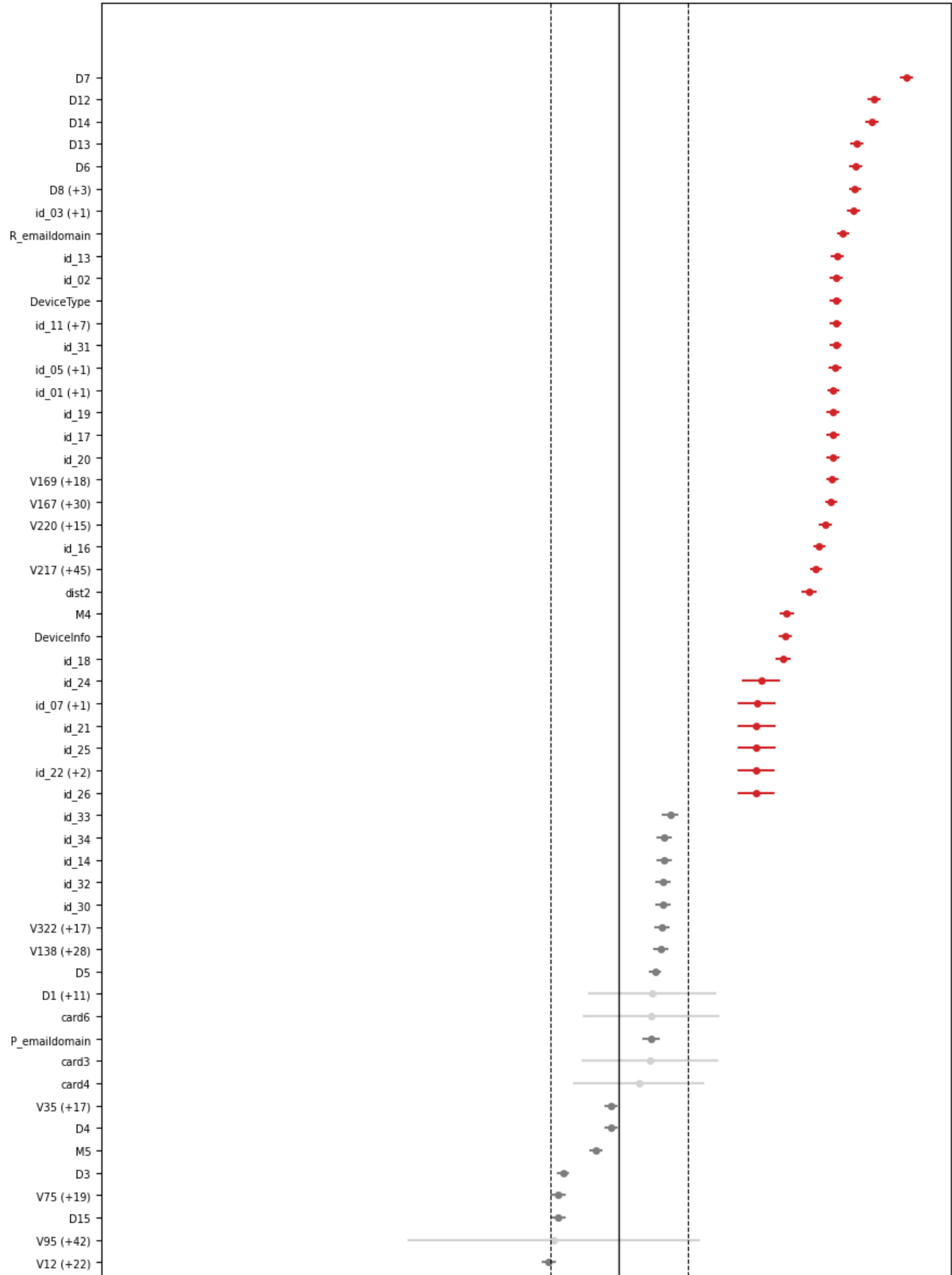
Find it: `# D0 (H4): fraud rate when missing vs present, for every missingness pattern, then the cell starting from scipy.stats import chi2_contingency for the statistical test.`

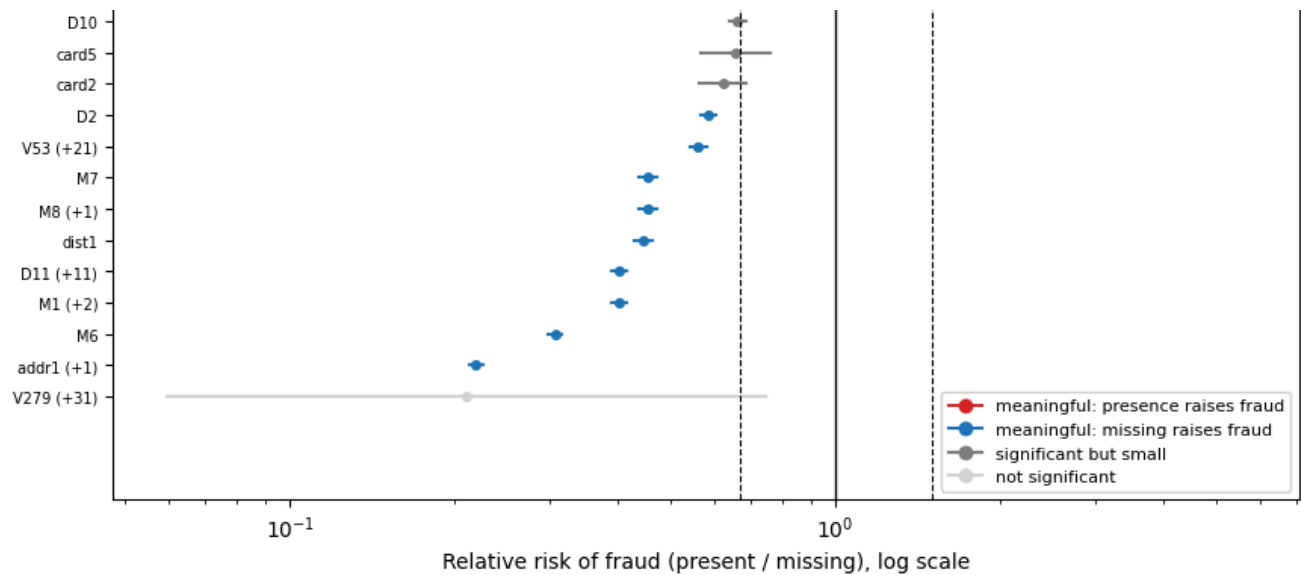
For each of the 67 missing-data patterns, we compared fraud when the columns are present vs missing. **42 of 67 made a meaningful difference.** In 33, fraud is higher when data is present; in 9, when it's missing.

19 more were real but small, and 6 weren't significant. "Meaningful" means two things at once: the difference passes a statistical test (corrected for running 67 tests), **and** its whole confidence interval is beyond $1.5\times$ or $0.67\times$.

Find it: # Forest plot: relative risk with 95% CI per missingness pattern

H4: Fraud relative risk by missingness pattern (95% CI)



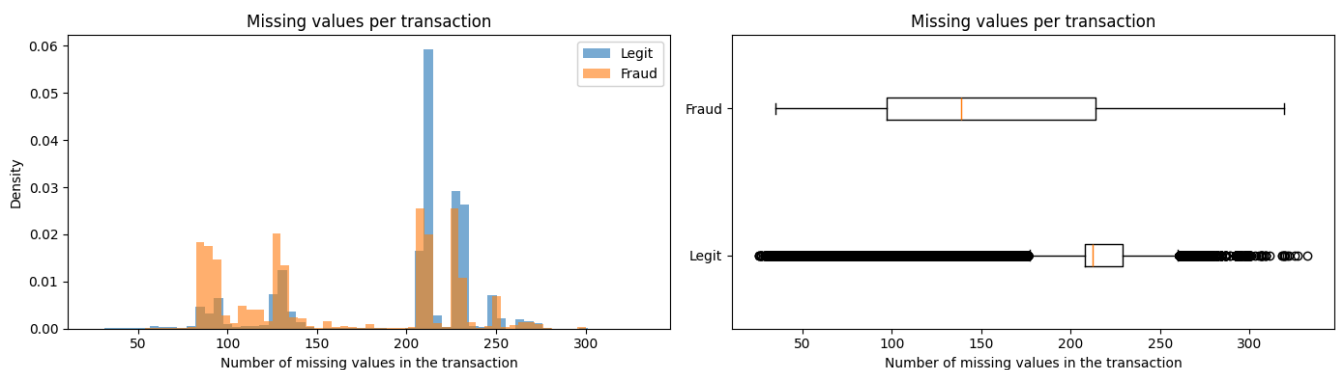


How to read it. Each row is one pattern. The dot is the ratio (fraud when present ÷ fraud when missing) and the line is its 95% confidence interval. The solid line at 1 means "no difference"; the dashed lines at 0.67 and 1.5 are our "meaningful" cutoffs.

- **Red rows** (right of 1.5): fraud rises when the data is present. The strongest is **D7** : 15.0% fraud when recorded, 2.7% when missing, even though it's 94% empty.
- **Blue rows** (left of 0.67): fraud rises when the data is missing. The biggest is the address: 11.4% fraud when missing vs 2.5% when present.
- **Gray rows** are real but small, or not significant. The non-significant ones all have tiny groups, which is why their lines are so long.

A whole cluster of identity columns sits at about 3.5. That's one signal ("does this transaction have an identity record?") showing up many times.

Find it: # Missing values per transaction: fraud vs legit



How to read it. How many of the 434 fields each transaction is missing. Fraud is surprisingly more **complete**: the median fraud transaction is missing 139 fields, compared with 212 for legit ones.

So what? Never drop a column just because it's mostly empty. Tree models keep missing values as they are, which is what we want.

The identity effect, and our first product-type surprise

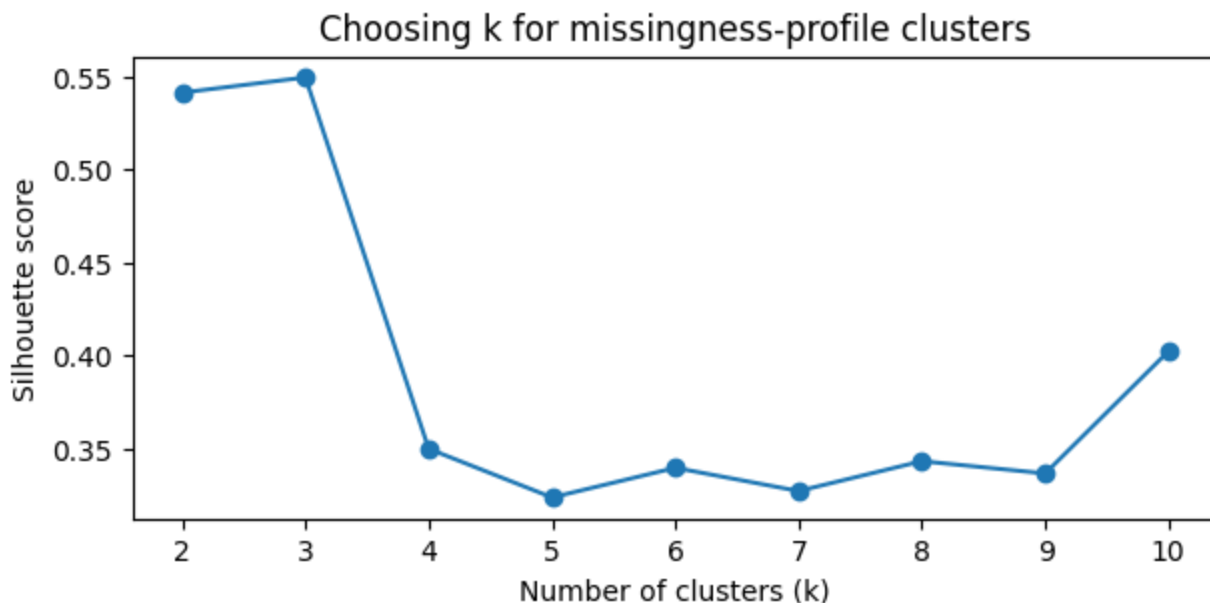
Find it: # D0 (H3): identity coverage and fraud rate with vs without identity, then `c = train_df[train_df["ProductCD"] == "C"]` for the within-product check.

Transactions with identity data have 7.55% fraud vs 2.13% without, which is 3.5× on the surface. But product `W` never has identity data and has the lowest fraud rate. Compare within product `C`, where both kinds exist, and the effect is **2.0×** (95% CI 1.8 to 2.3). That's the real number.

Three kinds of transactions

There are 11,341 missing-data profiles, but 118 cover 80% of transactions. Clustering them (without telling the algorithm about product or identity) found three groups:

Find it: the cell starting `from sklearn.cluster import MiniBatchKMeans` (choosing `k`), then `km = MiniBatchKMeans(n_clusters=best_k, ...)`



How to read it. The silhouette score measures how well separated the clusters are (higher is better). It peaks at `k = 3` (about 0.55), so we used three clusters.

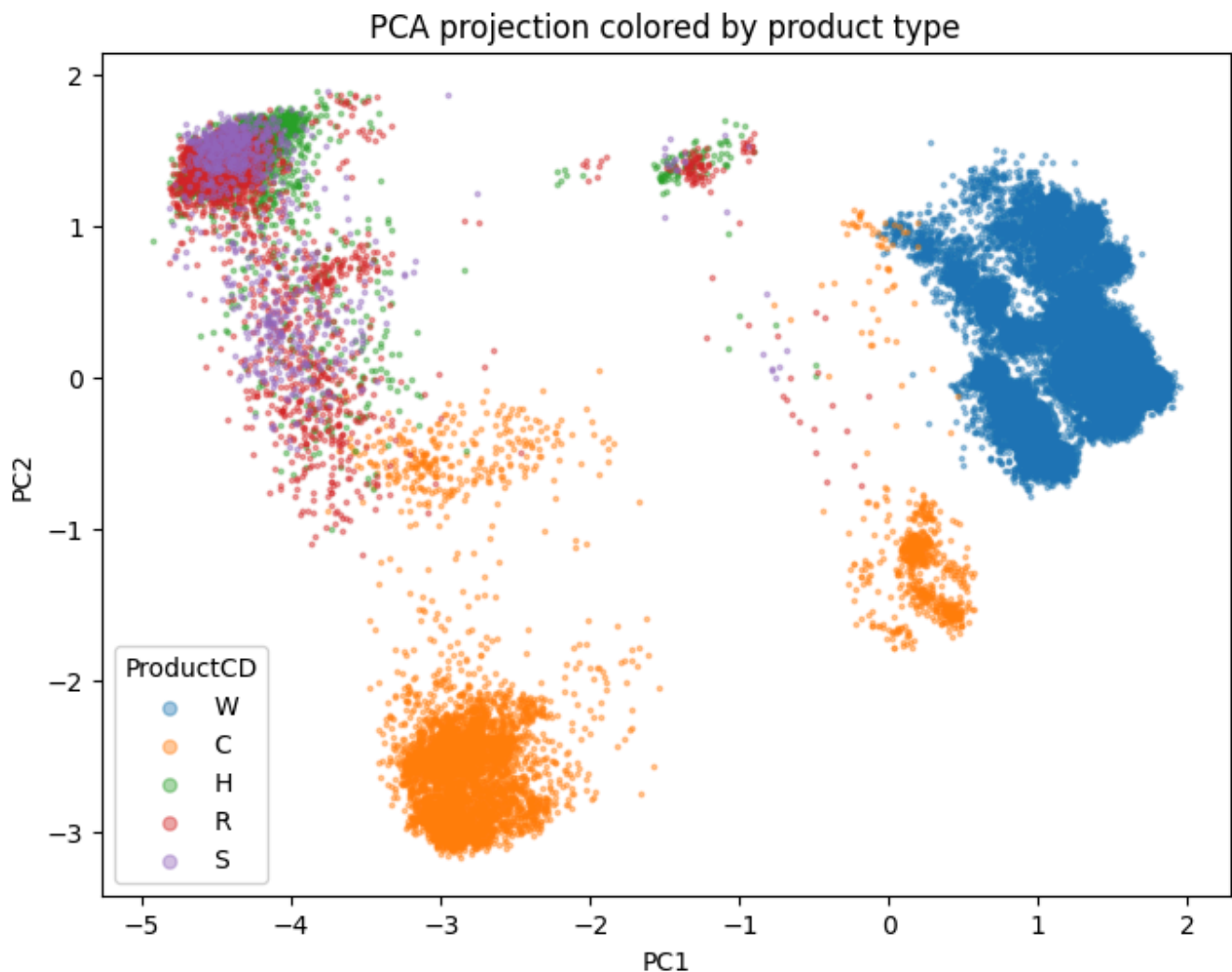
Group	Share	What it is	Fraud rate
1	75%	No identity data, almost all product <code>W</code>	2.1%

Group	Share	What it is	Fraud rate
2	14.5%	Identity data, products H , R , S	4.4%
3	10.6%	Identity data, product C	12.0%

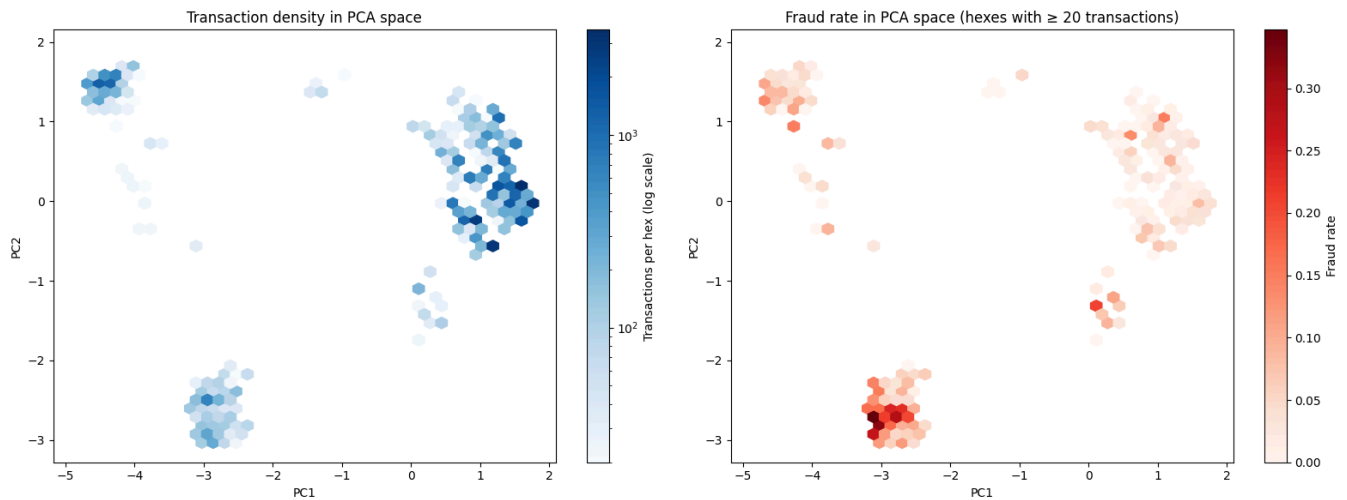
The clustering never saw product or identity, yet it recovered both. **Product type is the backbone of the data's structure.**

We also compared rare vs common profiles. Rare profiles look riskier overall (7.1% vs 2.6%), but within products the difference shows up only in product **C** (14.6% vs 7.2%).

Find it: `pca = PCA(n_components=2, random_state=42)` and the plotting cells after it.



How to read it. Each dot is a transaction, placed by its missing-data profile after squeezing 67 columns into 2. Colors are product types. Products separate cleanly on their own: **W** forms several lobes, **C** sits apart, and **H** , **R** , **S** overlap almost completely.



How to read it. Same map, split into hexagons. Left: how many transactions land in each hexagon. Right: the fraud rate inside each (only hexagons with 20+ transactions). The product **C** identity area is genuinely risky (core hexagons at 25 to 30% fraud), while most of product **W** stays under 5%.

Product type: rate vs volume

Find it: `prod = train_df.groupby("ProductCD")["isFraud"].agg(...)`

Product	Fraud rate	Share of all fraud
C	11.3%	38.5%
S	6.2%	3.0%
H	4.6%	8.3%
R	3.6%	7.0%
W	2.1%	43.1%

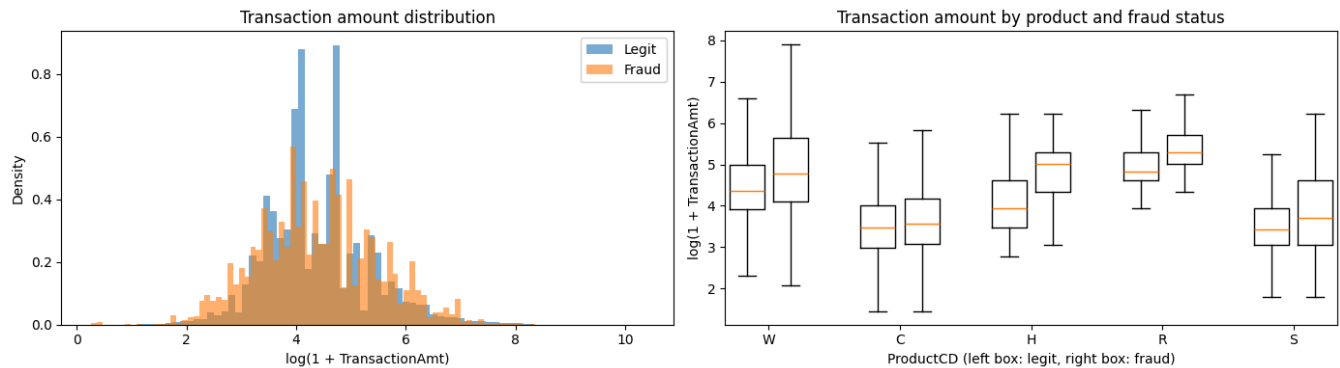
Product **W** is the "safest" per transaction, yet it holds the most fraud because it's 73% of all transactions. A model that only nails product **C** would still miss most fraud.

Branch 3: Which features actually predict fraud?

Four signals hold up after we check within each product: **hour of day, card type, device type and amount**. Email helps only in some products. Because product type fooled us in Branch 2, every check here is done product by product.

Amount: fraud spends more, not less

Find it: # D0 (Branch 3, H1): transaction amount by fraud status, overall and within product



How to read it. Left: the spread of amounts (log scale), fraud vs legit. Legit amounts spike at a few common price points; fraud is more spread out. Right: box plots per product, legit on the left of each pair, fraud on the right. In **every** product the fraud box sits higher.

Product	Legit median	Fraud median
H	\$50	\$150
R	\$125	\$200
W	\$78	\$117
S	\$30	\$40
C	\$31	\$35

Settling a team debate. Eric found that payments of \$10 or less are fraud 7.2% of the time, vs 2.9% for \$30 to \$100, which suggested card testing. We reran his buckets product by product:

Find it: # D0 (B3-H1 follow-up): fraud rate by amount band, overall and within product

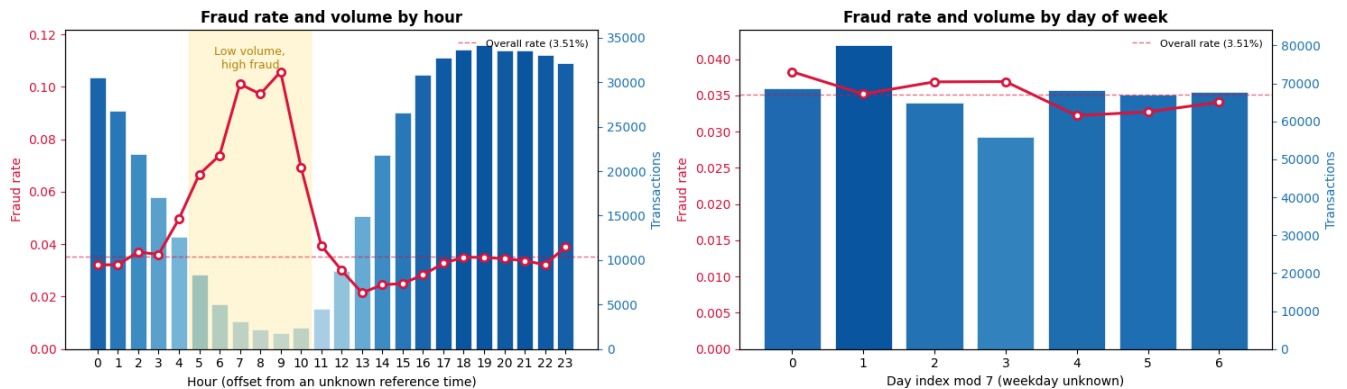
Amount	Product C	Product W
Up to \$10	8.0%	1.3%
\$10 to 30	10.7%	1.2%

Amount	Product C	Product W
\$30 to 100	12.0%	1.6%
\$100 to 500	14.6%	2.5%
Over \$500	too few	4.4%

Inside each product, fraud **rises** with the amount. The small-payment spike was product mix: 78% of those tiny payments are product **C**, the riskiest product.

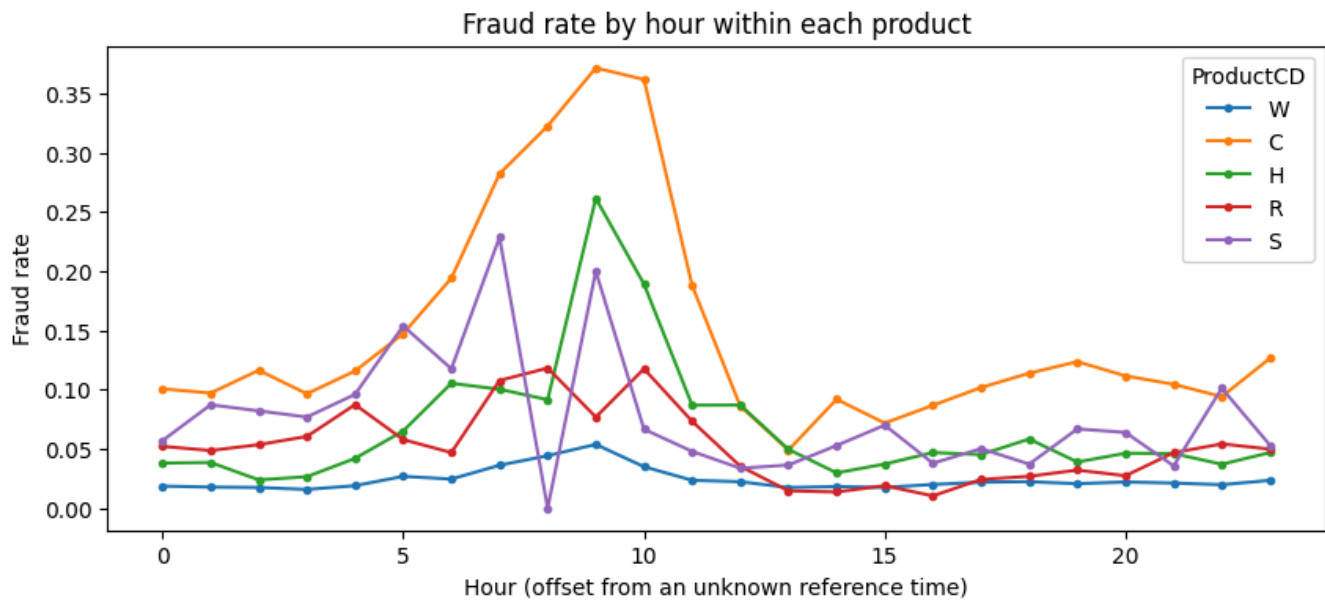
Hour of day: the most consistent signal

Find it: the cell starting `import matplotlib.colors as mcolors`



How to read it. Bars are transaction volume (right axis); the red line is the fraud rate (left axis); the dashed line is the 3.51% average. Left chart: in hours 5 to 10 (yellow band), volume collapses and fraud jumps to around 10%. Right chart: day of week barely moves (3.2% to 3.8%).

Find it: `hour_prod = train_df.pivot_table(index="hour", columns="ProductCD", ...)`



How to read it. One line per product. Every line bumps up in the same quiet hours, so this is not product mix. Product **C** peaks near 37%; product **W** rises more gently, but it's still more than 2.5× its usual rate.

Find it: `night = train_df["hour"].between(5, 10)`

Product	Fraud, hours 5 to 10	Fraud, other hours	Ratio (95% CI)
W	3.3%	2.0%	1.64 (1.50 to 1.80)
C	20.6%	10.5%	1.97 (1.86 to 2.09)
H	10.7%	4.3%	2.51 (2.17 to 2.90)
R	7.4%	3.5%	2.13 (1.73 to 2.63)
S	13.0%	5.9%	2.23 (1.69 to 2.93)

Across products, fraud in hours 5 to 10 is **1.6× to 2.5×** the other hours, and every confidence interval clears 1.5. One honest caveat: we picked that window after looking at the chart, so the true effect may be a bit smaller.

Card type: credit is riskier

Find it: `# D0 (B3-H3): fraud rate by card network and card type`

Credit cards have 6.73% fraud vs 2.40% for debit. It holds where it counts: **2.8×** in product **W** and **2.1×** in product **C**, which together hold 82% of fraud. Visa and Mastercard are nearly identical. Discover looks risky, but only in product **W** (8.2% vs about 2% for the others).

Email: mostly product mix, with a few real signals

Find it: # Within-product check for the top purchaser providers and # Within-product check for internet-provider domains

- **Hotmail** looked risky overall (5.2%), but over half its transactions are product **C**. Within **C** it's actually **below** average (7.6% vs 11.3%).
- **Gmail and Outlook** run above average inside products **C**, **H** and **R**. Inside product **W**, the provider barely matters.
- **Internet-provider emails** (att.net, sbcglobal.net, verizon.net) are genuinely low-risk, even within product **W** (0.5% to 0.8% vs 2.1%).
- **Matching purchaser and recipient emails** looked 4.2× riskier, but that's mostly product **C** in disguise. It's real only in products **H** and **R**.

Device: mobile is riskier

Find it: # D0 (B3-H5): device information among transactions with identity data

This only covers transactions with identity data (no product **W**). Mobile beats desktop for fraud in **every** product: 3.4× in **R**, 2.9× in **S**, 1.3× in **H** and 1.2× in **C**.

Apple devices look safe overall, but they never appear in product **C**, so that's partly product mix. Inside **C**, every Android family is riskier than Windows (9.5%): Huawei 17.8%, Motorola 16.3%, Samsung 14.4%. Less common Android devices are the riskiest sizable group overall at 18.5%.

Branch 4: Does fraud stick to accounts?

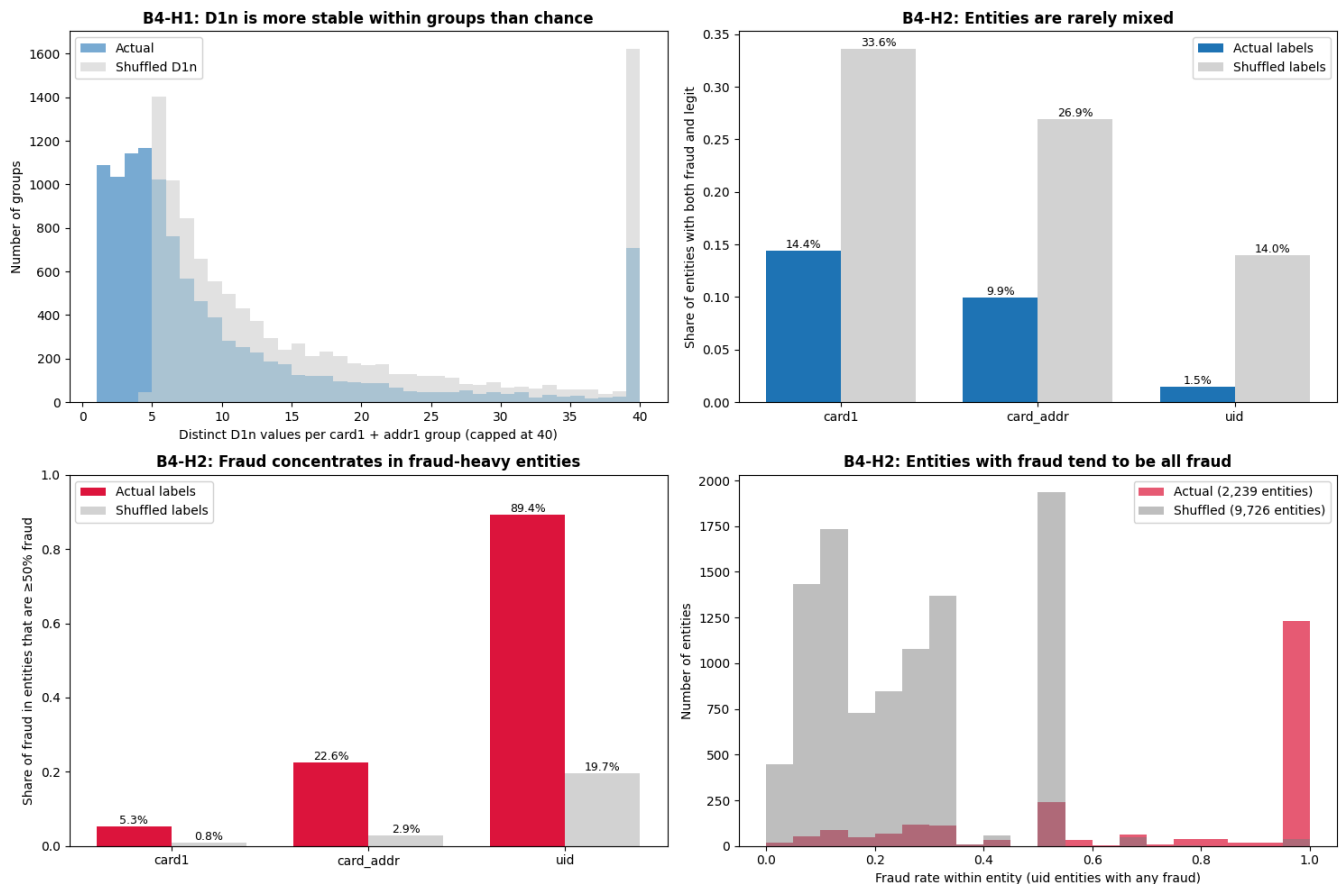
Yes, very strongly. Among repeat accounts with a known address, **89% of fraud** sits in accounts that are at least half fraud. With random labels it would be 20%.

Why we checked. The dataset's labeling rule: once a card is reported for fraud, its later linked transactions get labeled fraud too. So fraud should cluster by account. There's no account ID, so we used our homemade **uid** (card identifier + address + estimated card start day).

A fix we made along the way. At first, every transaction with a missing address on the same card identifier shared the text "nan", so unrelated purchases got merged into fake accounts. Now each of those 53,761 transactions (11.4%) gets its own marker. That removed the fake accounts, and the clustering went from 77% to 89%.

The trick behind every comparison: shuffling. We scramble which transactions are fraud while keeping every account the same size. That shows what the numbers look like by pure chance. The gap between "real" and "shuffled" is the real effect.

Find it: # D0 (B4-H1), # D0 (B4-H2), and the plotting cell starting `fig, axes = plt.subplots(2, 2, figsize=(15, 10))`



Top left: is our card start day meaningful? We grouped transactions by card identifier and address (groups with 5+ transactions) and counted the distinct start days in each group. Blue is real, gray is shuffled. Real groups pile up on the left: a typical group has 5 start days vs 11 shuffled. So `D1n` tracks real cards. It isn't perfect, though: only 10% of groups have a single start day, and some have hundreds (the spike at 40 is everything capped at 40). That's why we call `uid` an **approximate** account.

Top right: how often do accounts mix fraud and legit? Rarely. With `uid`, only 1.5% of accounts contain both, vs 14.0% if labels were random. The finer the key (from `card1`, to `card + address`, to `uid`), the purer the accounts.

Bottom left: where does the fraud sit? The share of all fraud in accounts that are at least half fraud. With `uid` it's 89.4% real vs 19.7% shuffled. This is the headline number.

In code, it's this line inside `entity_purity`:

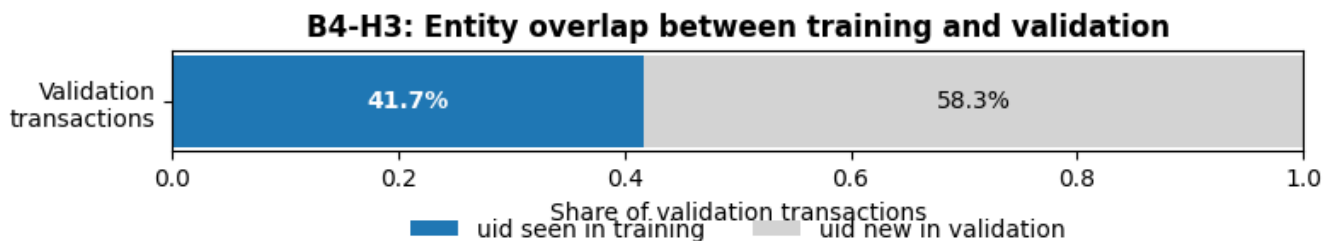
```
frauds[frauds / sizes >= 0.5].sum() / frauds.sum()
```

A mini example of that calculation: four accounts with 5 of 5, 3 of 4, 1 of 10 and 1 of 6 transactions fraud. There are 10 fraud transactions in total, and 8 of them sit in the first two (mostly-fraud) accounts, so the result is 80%.

Bottom right: accounts with any fraud. For every account containing at least one fraud, how much of it is fraud. Real accounts (red) pile up at 100%: about 1,200 accounts are entirely fraud. Shuffled (gray) sits mostly under 50%. Real fraud is concentrated in 2,239 accounts; random labels would spread it across 9,726.

One caveat on the 89%. Only accounts with two or more transactions count, and each address-missing transaction is now its own one-transaction account. So the 89% describes fraud in repeat accounts with a known address, not the address-missing group (which itself has 11.4% fraud).

Find it: # D0 (B4-H3): entity overlap between training and validation periods



Do accounts carry over? 41.7% of validation transactions come from an account already seen in training. That's a slight undercount: an address-missing transaction now gets a unique marker, so it can never match. The test set starts a month later, so its overlap is probably lower.

So what? Account-level features are likely our strongest weapon: how many transactions an account has made, its usual amount, how unusual this purchase is for it. But two traps:

- **Don't feed the raw `uid` to the model.** It would memorize accounts instead of learning patterns.

- **Don't use an account's past fraud labels** unless they'd really be known at prediction time. Chargebacks arrive weeks later, so that would be cheating.

What we're taking into modeling

Use tree models, keep missing values, judge by PR-AUC, and report results by product and by new vs known accounts.

	What we'll do
Model	Tree models like LightGBM or XGBoost. They handle missing values and learn product interactions on their own.
Features in	Product, amount, hour of day, card type, card network, email provider, identity flag, device type and family, label-free account summaries
Worth trying	Amount compared with the product's usual amount, rare-profile flag, missingness cluster
Features out	Raw time or day, the raw account key, account fraud history we wouldn't know yet
V columns	The trimmed set of 210 for simpler models; optional for trees
Scoring	PR-AUC and ROC-AUC, never accuracy
Validation	Keep the time split, and consider a few weeks' gap to mimic the month before test
Reporting	Product W vs C separately, and known vs new accounts separately

And always: learn anything from data on the training set only, then apply it unchanged to validation and test.

Find it: `pd.DataFrame(decision_log)` in the last cell prints every decision the notebook logged.

Things we're still unsure about

- **Our account key is approximate.** D1n is noisy in some groups, and transactions with a missing address (11.4%) are left out of the account-clustering result, so we don't know how their fraud clusters.
- **The night-time window was picked by eye.** Confirm the hours 5 to 10 effect on validation.
- **Validation may be too kind.** It starts the day after training, while test starts a month later.
- **Masked columns.** We can show that D7 matters, but not always why.

Mini glossary

- **Fraud rate:** the share of transactions that are fraud.
- **Relative risk (ratio):** one group's fraud rate divided by another's. 2 means twice as common, 0.5 means half, 1 means no difference. It shows association, not cause.
- **95% confidence interval:** the range of ratios the data supports. If the whole range is above 1 (or below 1), the difference is reliable.
- **Product mix:** when something looks linked to fraud only because it's more common in a high-fraud product. Checking within each product removes it.
- **Benjamini-Hochberg correction:** when you run many tests at once, some look significant by luck; this adjusts for that.
- **Spearman correlation:** measures whether two columns rise together, even if not in a straight line.
- **k-means and silhouette score:** k-means groups similar rows; the silhouette score (-1 to 1) says how well separated the groups are.
- **PCA:** compresses many columns into a few summary numbers, here just to plot them.
- **Shuffled baseline:** scramble one column while keeping everything else fixed, to see what "no real relationship" looks like.